

PRAKTIKUM SISTEM OPERASI

MODUL 11
(MEMORI XINU)



Oleh:

(Faishal Arif Setiawan)

(2311104066)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026

JURNAL/TP

1. Soal 1

[80 poin] Buatlah perintah baru bernama **freememory** yang memiliki dua fungsi berikut:

- [40 poin] Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
- [40 poin] Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

Jawaban:

```

      /\  /\  \  _ | | _ | \  | | | | | |
     /  /  \  \  _ | | _ | \  | \  | \  -- /
    --  --  --  --  --  --  --  --  --
-----2311104066-----

Welcome to Xinu!

xsh $ freememory

Free List:
Block address    Length (dec)    Length (hex)
-----
0x0018E110       -975128\t      0xFFFF11EE8
0x00100000        5070848\t      0x004D6000
0x005D8000        122880\t       0x0001E000

Total FreeSpace: 4218600
xsh $ xsh $ xsh $ xsh $ xsh $ xsh $ _
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

2. Soal 2

[4 poin per subsoal] Jawablah pertanyaan berikut:

- Mengapa Xinu memisahkan data segment dan BSS segment?
- Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?
- Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?
- Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?
- Apa tantangan utama dalam penggunaan heap di Xinu?

Jawaban:

- a. Xinu memisahkan data segment dan BSS segment untuk meningkatkan efisiensi penggunaan memori serta mempermudah proses pengelolaan saat program dijalankan. Data segment digunakan untuk menyimpan variabel global yang telah diinisialisasi, sedangkan BSS segment menyimpan variabel global yang belum diinisialisasi. Dengan pemisahan ini, variabel yang belum memiliki nilai awal tidak perlu disimpan secara eksplisit di dalam file executable, sehingga ukuran program menjadi lebih kecil dan penggunaan memori menjadi lebih optimal.
- b. Selama eksekusi sistem, proses alokasi dan dealokasi memori sangat memengaruhi ukuran free space yang tersedia. Ketika suatu proses meminta memori, sistem akan mengalokasikan sebagian dari free space sehingga jumlah free space akan berkurang. Sebaliknya, ketika memori yang digunakan oleh proses dilepaskan atau didealokasikan, maka free space akan bertambah kembali. Proses ini berlangsung secara dinamis tergantung pada jumlah dan kompleksitas proses yang berjalan, sehingga ukuran free space dapat berubah-ubah selama sistem beroperasi.
- c. Penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack karena pengelolaannya dilakukan secara manual oleh programmer. Pada heap, memori yang telah dialokasikan harus dibebaskan secara eksplisit, sehingga jika programmer lupa melakukan dealokasi, dapat terjadi memory leak. Selain itu, heap juga rentan terhadap fragmentasi memori akibat alokasi dan dealokasi yang tidak teratur. Berbeda dengan stack yang dikelola secara otomatis oleh sistem dengan mekanisme LIFO, sehingga lebih terstruktur dan kecil kemungkinan terjadi kesalahan pengelolaan memori.
- d. Xinu menggunakan struktur linked list untuk menyimpan free block karena fleksibel dalam menangani ukuran blok memori yang bervariasi. Dengan linked list, setiap blok memori yang kosong dapat direpresentasikan sebagai node yang berisi alamat awal dan ukuran blok tersebut, serta pointer ke blok berikutnya. Struktur ini memudahkan proses penambahan, penghapusan, dan penggabungan blok memori saat terjadi alokasi maupun dealokasi, sehingga manajemen memori menjadi lebih efisien dan dinamis.
- e. Tantangan utama dalam penggunaan heap di Xinu adalah potensi terjadinya fragmentasi memori, kesalahan dalam pengelolaan memori, serta kemungkinan kehabisan ruang memori. Fragmentasi terjadi ketika ruang memori kosong

terbagi menjadi bagian-bagian kecil yang sulit digunakan kembali secara optimal. Selain itu, karena heap harus dikelola secara manual, terdapat risiko memory leak jika memori tidak dibebaskan dengan benar. Tantangan lainnya adalah memastikan bahwa alokasi memori tidak melebihi kapasitas free space yang tersedia, yang dapat menyebabkan kegagalan dalam proses alokasi.